PolicyC: Request-Conditioned Compilation of Large Policy Prompts

B. Garcia

September 2026

Abstract

Large language-model system prompts increasingly behave like policy programs: they combine safety constraints, privacy rules, tool requirements, artifact procedures, confirmation protocols, and response conventions. Supplying the entire policy on every request increases context and cost, while naive compression can remove a rare but decisive obligation. We study whether a large prompt P and request x can be compiled into a much smaller active slice P_x such that a model using P_x preserves the critical obligations satisfied by the same model using P.

We present PolicyC, a research prototype built around a validated graph of 43 structured policy nodes across six domains. A deterministic TypeScript compiler performs request and artifact-context matching, conservative retention, dependency closure, predicate-driven specialization, and compact prompt emission. A separate Python `asyncio` runtime executes paired full-policy and compiler-slice trials with bounded concurrency, hard call/token/cost ceilings, atomic raw-response persistence, resumability, hash-linked provenance, a rebuildable SQLite catalog, deterministic diagnostics, and strategy-blind semantic review.

We report five frozen held-out studies using a pinned GPT-5 mini snapshot: 280 cases, 1,680 model executions, and $4.901 in recorded cost. Compiled prompts reduced mean actual input by 89.69–98.23% and uncached-equivalent cost by 55.36–67.80%. However, conditional critical-obligation preservation ranged from 75.76% to 86.49%, below the preregistered 95% target in every study. Compiler 0.7 lost 15 of 33 regressions to one defect: a conditional confirmation rule emitted as an unconditional instruction. Compiler 0.8 added a specialization stage that resolves that rule from confirmation state already present in the request. It repaired all six spent development cases and matched none of the four fresh confirmation cases in the next held-out set, whose authors expressed authorization as "no need to check back" and "go ahead" rather than "I confirm." Its preservation was 75.76% (Wilson 95% interval: 67.79–82.27%) on a set whose brief deliberately probed tool limits and confirmation state; the two studies use different cases and are not a controlled comparison. Its largest regression class was calling a tool the user had asked not to use.

Compiler 0.9 rebuilt the pipeline as a compiler: a typed request state read once by a frontend, declared conditional branches on policy nodes, a declared precedence table, and a printer; its extractor frontend, one model call per request at compile time, read 6 of 6 fresh authorization phrasings where the regexes read 0. It preserved 75.91% on a fifth held-out set, within a point of 0.8. Every regression classified into a compiler bin: authorization was read correctly on all of them, and the losses were a selector with no marker for current information stated in plain speech, a branch derived from one spent case that fired on every fully specified artifact edit lacking a field contract, over-read limits, and one class needing a fact the state does not carry.

The experiments therefore support a mixed conclusion. Request-specific policy compilation reliably yields large efficiency gains in this synthetic system, but neither text selection, pattern-matched specialization, nor a compiler with a model-assisted reader preserves critical behavior well enough to replace the full prompt. The negative results localize the central research problem: reading the request is now the smaller part; representing what the policy requires of each situation, and proving that the request satisfies it, is the larger.

1 Introduction

Production system prompts are growing from short behavioral instructions into heterogeneous policy bundles. A single prompt may state when live research is mandatory, which sources are authoritative, how destructive actions must be confirmed, which personal attributes must not be inferred, what tools may be called, how files must be inspected, and which response format is appropriate. Most user requests activate only a fraction of these rules, yet conventional inference supplies the full prompt for every request.

This creates an appealing systems question: can a request-conditioned compiler activate only the policies relevant to the current request? A successful compiler could reduce input tokens, cost, and latency while making the active contract easier to inspect. The failure mode, however, is asymmetric. Dropping redundant prose is harmless; dropping one confirmation, privacy, or tool-integrity rule can be severe. Average semantic similarity is therefore an inadequate target.

Generic prompt-compression methods remove low-information passages or tokens while preserving downstream task accuracy [1, 2, 3, 4]. Policy compilation has a different objective. It must preserve obligations whose importance is conditional on a specific request, tool surface, artifact, or risk. Related alignment work studies explicit principles and privileged instruction levels [5, 6], but does not directly test request-conditioned removal of privileged policy text.

We formalize the problem as follows. Let P be a policy program, x a user request plus optional structured context, and S a selector. The compiler emits

$$P_x = \mathrm{closure}(S(P, x)), \tag{1}$$

where closure adds every transitive policy dependency. Let $C(P, x, y)$ indicate that answer y, produced under policy condition P, satisfies all independently specified critical obligations for x. The primary target is conditional preservation:

$$\theta = \Pr[C(P_x, x, Y_{P_x}) = 1 \mid C(P, x, Y_P) = 1]\,. \tag{2}$$

Conditioning on full-policy success is essential. If both conditions fail, the pair does not show that compilation preserved a successful obligation-bearing behavior. The primary efficiency target is

$$R_{\mathrm{input}} = 1 - \frac{\mathbb{E}[T_{\mathrm{input}}(P_x, x)]}{\mathbb{E}[T_{\mathrm{input}}(P, x)]}. \tag{3}$$

This paper makes six contributions:

1. a typed representation of policy nodes, triggers, dependencies, obligations, and prohibitions;
2. a deterministic request-conditioned selector and dependency-closed prompt emitter;
3. a specialization stage that swaps a node's emitted branch when an authored predicate is satisfied by the request, with a fail-closed default and a recorded evidence trace, and its successor: a typed request state, declared branches, a declared precedence table, and a swappable frontend whose model-assisted extractor is persisted and hashed into the run's identity;
4. a safety-bounded paired-experiment runtime with complete paid-call accounting and resumability;
5. five frozen held-out studies that separate efficiency from behavioral preservation; and
6. a failure taxonomy showing that conditional semantics, explicit user limits, and instruction precedence, rather than graph closure alone, dominate the remaining errors; that a surface-form predicate for one of them did not generalize; and that once the request is read correctly, the losses move to selection and to conditions the policy representation does not yet declare.

2 Related Work

Prompt compression. Selective Context filters prompt content using self-information [1]. LLMLingua introduces budget controllers and token-level compression for efficient inference [2]; LongLLMLingua adapts compression to long-context retrieval and position bias [3]; LLMLingua-2 frames task-agnostic compression as token classification distilled from a larger model [4]. These methods primarily evaluate answer quality, retrieval, or semantic fidelity. PolicyCinstead treats policies as typed program elements and evaluates independent obligations because infrequent policy text may be critical even when it appears semantically peripheral.

Principle-guided behavior and instruction priority. Constitutional AI demonstrates how an explicit written constitution can guide critique and preference learning [5]. The Instruction Hierarchy formalizes priority among system, developer, user, and tool-originated instructions and improves resistance to lower-trust conflicts [6]. PolicyCdoes not train a model or introduce a new hierarchy. It asks which privileged instructions can be omitted for one request without changing critical behavior.

Partial evaluation and program slicing. The compiler analogy is closer to partial evaluation than to generic summarization. A partial evaluator specializes a program using known inputs while preserving its semantics [7]. Program slicing retains statements that may affect a criterion. Policy prompts are not conventional programs, and language models are stochastic interpreters, but the analogy motivates explicit dependencies, conservative closure, and paired semantic tests. The experiments show where the analogy breaks: flattening a conditional policy into unconditional natural language can change behavior even when the correct node was selected.

Behavioral evaluation. Structural recall and behavioral preservation are distinct. A selector can retrieve every annotated node while the emitted wording weakens, strengthens, or misconditions those rules. PolicyCtherefore never treats its own selected nodes as behavioral ground truth. Each case carries independently declared obligations, and primary results come from strategy-blind answer comparison rather than selector metrics alone.

3 System Design

3.1 Policy representation

The current synthetic enterprise-agent prompt is represented as 43 manually structured nodes in six YAML packs: core behavior, current information and web use, files and artifacts, email and calendar, images, and writing. The graph grew from 39 nodes in compiler 0.5 to 43 in compiler 0.7 as held-out failures were converted into development requirements.

Each node contains a stable identifier, kind, severity, priority, triggers, optional dependencies, model-visible runtime instructions, obligations, prohibitions, and validator references. Node kinds distinguish universal rules, request-gated rules, and structural definitions. Zod runtime validation rejects unknown fields and invalid enum values. Graph validation rejects duplicate identifiers or edges, missing references, self-dependencies, cycles, invalid always-active nodes, unknown validators, and unreachable structural nodes.

The representation is intentionally explicit. The current system compiles a structured policy graph; it does not yet parse an arbitrary natural-language system prompt into policies automatically.

3.2 Selection and dependency closure

Given request x, artifact metadata, and the tool surface, the TypeScript compiler:

1. detects request intents using deterministic lexical rules;
2. activates universal policies;
3. matches triggers against request text, artifact type, operation, features, domain hints, risk hints, and tools;
4. conservatively retains high-impact safety, privacy, and tool-integrity rules;
5. traverses the transitive `requires` graph; and
6. orders selected nodes deterministically.

The closure operation is exact with respect to declared graph edges. This establishes dependency completeness, not behavioral completeness: an undeclared dependency or a lossy emitted instruction can still fail.

3.3 Prompt emission

The compiler serializes five experimental strategies: `full_policy`, `compiler_slice`, `kernel_only`, `direct_matches`, and `conservative_expanded`. The held-out studies compare only `full_policy` and `compiler_slice`.

Compiler 0.5 introduced a compact universal kernel to avoid repeatedly emitting honesty, privacy, hidden-reasoning, fabrication, trace-exposure, tool-simulation, and background-work policies. Compiler 0.6 removed a leaked internal tool-status directive and added policies derived from confirmed v0.5 regressions. Compiler 0.7 further hardened tool availability and selected regressions from v0.6. Despite those changes, v0.7 revealed that the emitter still converts some conditional policies into unconditional actions.

3.4 Specialization

Compiler 0.8 inserts a specialization stage between dependency closure and emission. A policy node may declare a predicate and an authored *satisfied* branch consisting of an alternative runtime instruction and obligation set. After closure, the stage evaluates each selected node's predicate against the request and artifact context; when the predicate holds, the satisfied branch replaces the node's default text and obligations, and when it does not, the node is emitted exactly as authored. Selection is never altered, so selector metrics are unchanged across 0.7 and 0.8, and each artifact records a trace of every evaluated predicate with its outcome and evidence. The default is fail-closed: any unrecognized or ambiguous state leaves the conservative branch in place.

The single predicate in compiler 0.8, `explicit_confirmation`, holds when one sentence of the request contains the user's own first-person present-tense confirmation of the operation the context declares, is neither negated, conditional, future, quoted, nor reported, and names every field the source policy requires for that artifact and operation (for example recipient, body, and attachment scope for an email send). Seven confirmation-bearing nodes carry a satisfied branch that instructs the model to perform exactly the confirmed action without asking again. The predicate was implemented as bounded regular expressions over the confirmation sentence, tuned against six spent v0.7 cases and fifteen negative controls covering third-party, quoted, conditional, negated, and under-specified confirmations.

3.5 Intermediate representation and frontends

Compiler 0.9 replaces the specialization stage with a compiler in the ordinary sense. A *frontend* reads the request once into a typed request state: authorization (present, reported, conditional, absent), a user-stated limit on the turn, the deliverable the user allows, a purpose the policy refuses to serve and whether a permitted task stands beside it, a stated output format, which fields the policy requires for the declared operation and whether each is stated, and whether the operation is named or negated. The state is recorded in every artifact. Policy nodes declare *branches*, each a condition over that state with its own text and obligations; the first branch decided true replaces the node's default, and an undecided condition falls through to the conservative default. Precedence between what the request allows and what a node obliges is a declared table over obligation types (a tool call yields to the user's limit; inspection also yields to asking first and to a refused purpose; a node the source policy mandates keeps its obligation under a user limit), applied by one masking loop and recorded per withheld obligation. When the resolved program asks for confirmation, the first emitted rule names the acting tools that must wait. The emitter prints the resolved program and decides nothing. Under the deterministic frontend the refactor changed no byte of any prompt on 273 spent cases; the two failure classes that had no representation, stated output format and asynchronous work, then received branches and a node.

Two frontends produce the state. The deterministic frontend is compiler 0.9's regex readers; on 23 blind paraphrase fixtures it recognized 1 of 6 fresh authorization phrasings and 1 of 5 fresh limits. The extractor frontend is one strict structured-output call per request at compile time (`gpt-5-mini-2025-08-07`, instructions written from the state's documented semantics, a schema derived from the same code that validates persisted reads), whose output is persisted, hashed, and named in the run manifest so that a run's identity fixes its reads; it recognized 6 of 6 and 5 of 5 on the same fixtures after two prompt revisions diagnosed against them. A read of *present* beside a disqualifier the request states in plain words (a negated review, a conditional clause) is capped to the disqualifier's state, so the extractor can only move a decision toward asking. A required field the read does not name counts as missing. Field contracts existed for email and calendar operations only.

3.6 Cross-language boundary

TypeScript is the single authority for policy loading, validation, matching, graph closure, candidate construction, prompt emission, hashing, and artifact serialization. Python owns provider calls, concurrency, rate limiting, timeouts, retry classification, atomic persistence, evaluation, reports, and run-catalog storage. Versioned JSON Schemas define the boundary, and Python verifies candidate and compiled-prompt hashes before execution.

Trial identifiers bind the run, case, sample, strategy, model, provider, and candidate. This prevents the behavioral runtime from silently reconstructing a different compiler.

3.7 Safe paid execution

The live runtime is designed to fail closed. A dry run validates every payload and produces expected and worst-case spend. Paid execution requires an exact run-specific confirmation string. Hard ceilings cover provider attempts, input tokens, output tokens, tool calls, and dollars. Missing usage is never silently treated as zero.

Raw provider responses are atomically persisted before parsing. Completed trials resume without another provider call when provenance matches. Ambiguous network outcomes consume a call and

reserved exposure and are not retried unless explicitly enabled. A local SQLite catalog stores run status, source commit, dataset and manifest hashes, model, trial counts, usage, costs, failures, and authoritative artifact paths. Immutable run directories remain the scientific source of truth and can rebuild the catalog.

The scheduler uses a bounded worker queue, an `asyncio` semaphore, and a sliding-window limiter. In the first 300-call held-out run, concurrency six completed the experiment in 8.35 minutes versus 49.12 minutes of summed provider-call latency, a 5.88× effective throughput ratio. This is an observed concurrency benefit, not a controlled sequential benchmark.

4 Evaluation Protocol

4.1 Case construction

Every case specifies the user request, artifact context, applicable obligations, critical obligation IDs, prohibitions, refusal expectation, tool expectation, and a semantic rubric. These labels are authored independently of the candidate prompt. Each case is executed under both strategies for the same sample index and pinned model. Dispatch order is deterministically counterbalanced.

Held-out sets v3 and v4 were written by three context-isolated authoring agents, each given only the authoring brief and the synthetic prompt and each responsible for a disjoint domain range; each author then audited another author's batch, and repairs were applied through a replayable transformation script until independent verification returned no high or medium findings. The v4 brief added one section, in the synthetic prompt's own words, asking each batch for cases on both sides of the prompt's confirmation and intent conditions: some where a careful reader would judge the action already authorized and some where a careful reader would still ask. It did not describe the compiler's predicate or its negative controls.

The studies use synthetic requests and a synthetic policy system. No private production data is included. Provider storage is disabled. The API key is loaded locally and excluded from run artifacts.

4.2 Outcome definitions

For a paired sample, four determinate critical outcomes are possible:

- both pass;
- full policy passes and compiler slice fails (FullOnly);
- compiler slice passes and full policy fails; or
- both fail.

The empirical conditional-preservation estimator is

$$\widehat{\theta} = \frac{n_{\text{both pass}}}{n_{\text{both pass}} + n_{\text{full only}}}. \tag{4}$$

We report trial-level Wilson 95% intervals and exact two-sided McNemar tests for discordant pairs. Because three generations from one case are clustered, these intervals and tests are descriptive and do not create three independent policy situations.

4.3 Deterministic and semantic evaluation

The deterministic evaluator checks case-local obligations plus a universal floor: no fake background work, hidden-reasoning disclosure, unsupported precision, raw tool traces, or simulated tool use. It

Table 1: Frozen held-out studies. "Pairs" counts complete paired outputs before semantic ungradables.

Compiler	Cases	Model calls	Complete pairs	Web searches	Cost	Output cap
0.5	50	300	129/150	0	$0.6881	2,048
0.6	50	300	139/150	41	$1.0357	2,048
0.7	60	360	180/180	20	$0.9333	3,072
0.8	60	360	163/180	16	$1.0653	2,048
0.9	60	360	177/180	26	$1.1790	3,072
Total	280	1,680	—	103	$4.9014	—

also scores refusal and tool behavior. These checks are reproducible but lexically brittle.

Primary behavioral results use strategy-blind semantic packets. Packets contain opaque answer IDs, the request, obligations, allowed or required tools, and observed strategy-neutral tool evidence. They omit strategy, prompt size, latency, tokens, and cost. Grade hashes are locked before answer IDs are joined to a private strategy map. Reviewers were isolated Codex reviewer agents, not human annotators; this limits external validity and is stated explicitly.

4.4 Frozen studies

Table 1 summarizes the five held-out executions. Each compiler was frozen before its corresponding case set was opened to candidate compilation. Once results were inspected, that dataset became development evidence for later versions and was never reused as fresh held-out evidence.

All studies use `gpt-5-mini-2025-08-07`, two strategies, three samples per case, and no retries; concurrency was six for v1–v3 and four for v4 and v5. Compiler 0.5 had no provider tools. Compilers 0.6 through 0.9 used web and synthetic function-tool cases, making their input and cost accounting harder but more representative of an agent policy surface. The v4 output cap was preregistered at 2,048 tokens because every completed v3 response had fit within it; 21 v4 executions (11 compiled, 10 full) nonetheless reached the cap, and the 17 pairs they affected were excluded from paired estimates and counted against the coverage gate, never as passes. Compiler 0.9 was compiled under the extractor frontend (Section 3.5): its 60 requests were read once by one model call each ($0.2678, reported beside the run, never netted) after the dataset froze and before the preregistration, and the reads file's hash is part of the run's identity. Three v5 trials were incomplete: two compiled responses to one case reached the 3,072 cap and one full-policy sample was starved by the call ceiling before any call.

5 Results

5.1 Efficiency

Table 2 shows the most stable result in the project. Every compiler reduced mean actual input by at least 89.69% and uncached-equivalent cost by at least 55.36%, and compiler 0.9 reproduced the pattern on a fifth independently authored set. Actual billed savings were smaller because the repeated full prompt benefited from prompt caching, while the compiled condition varied by request. Tool fees also matter: compiler 0.7 made 11 paid web searches versus nine under the full condition, and compiler 0.8 made six versus three over the complete pairs. Excluding search fees, 0.7's generation-only cost fell 19.75% and 0.8's 26.07%, but the preregistered total billed-cost reductions were 12.14% and 18.03%. Compiler 0.9 made 7 searches against the full condition's 15 over the complete pairs; its

Table 2: Compiled-condition changes relative to the full prompt. Positive reductions are favorable; positive latency deltas are slower.

Compiler	Input reduction	Billed-cost reduction	Uncached-cost reduction	Latency delta
0.5	98.23%	23.01%	67.80%	−19.95%
0.6	89.69%	24.50%	55.36%	+0.39%
0.7	93.75%	12.14%	59.46%	+14.63%
0.8	94.76%	18.03%	66.00%	−7.07%
0.9	92.99%	17.21%	58.69%	+13.36%

Table 3: Primary strategy-blind semantic results. Compiler 0.6 includes three ungradable pairs, so strategy pass-rate denominators differ.

Compiler	Full pass	Compiled pass	Both	Full only	Compiler only	Both fail
0.5	107/129	101/129	92	15	9	13
0.6	74/138	73/136	64	10	9	53
0.7	163/180	141/180	130	33	11	6
0.8	132/163	111/163	100	32	11	20
0.9	137/177	125/177	104	33	21	19

generation-only cost fell only 5.34% because the compiled condition produced 35% more output tokens, and the total billed reduction of 17.21% is carried by the search fees.

Latency did not improve consistently. Compiler 0.5 was 19.95% faster, compiler 0.6 was essentially unchanged, compiler 0.7 was 14.63% slower, compiler 0.8 was 7.07% faster, and compiler 0.9 was 13.36% slower. Output length, tool activity, cache state, and model reasoning all affect latency; input reduction alone does not guarantee wall-clock improvement.

5.2 Critical-obligation behavior

Tables 3 and 4 show that none of the held-out studies supports near-equivalence. Compiler 0.6 produced nearly identical marginal pass rates for full and compiled prompts, yet conditional preservation was only 86.49%. Equal averages can hide swapped successes: nine pairs passed only under the compiler while ten different pairs passed only under the full prompt.

Compiler 0.7 is a stronger negative result. The full prompt passed 90.56% of answers, whereas the compiled slice passed 78.33%. Conditional preservation fell to 79.75%, and 16 distinct cases contained at least one full-pass/compiler-fail result. The packet-level McNemar result is directionally significant, though a case-level sign test over 21 non-tied cases was $p = 0.0784$. Leave-one-case-out preservation ranged from 79.38% to 81.25%, indicating that no single case explains the result.

Compiler 0.8 on held-out v4 is the lowest preservation observed, 75.76%. The v3 and v4 sets differ, so this is not a controlled comparison with compiler 0.7 and no cross-version significance is claimed. The full prompt itself passed only 80.98% of v4 answers against 90.56% on v3, so v4 was harder for both conditions; its brief asked for explicit tool limits and both sides of confirmation state, which v3's did not. A case-clustered bootstrap over the 60 cases gives a 64.49–86.01% interval; 37 of 54 eligible cases were regression-free; the case-level sign test over 23 non-tied cases was $p = 0.0347$; leave-one-case-out preservation ranged from 75.19% to 77.52%. Preservation was flat across the three author domains (75.6–76.0%) and concentrated by tool: calendar and spreadsheet cases sat at 56.2%, gmail and no-tool cases at 82–83%.

Table 4: Conditional preservation and uncertainty. Intervals are trial-level Wilson 95% intervals and remain descriptive because samples are clustered by case.

Compiler	Preservation	Wilson 95%	Distinct full-only cases	McNemar p
0.5	92/107 = 85.98%	78.15–91.32%	9	0.3075
0.6	64/74 = 86.49%	76.88–92.49%	6	1.0000
0.7	130/163 = 79.75%	72.93–85.21%	16	0.0013
0.8	100/132 = 75.76%	67.79–82.27%	17	0.0019
0.9	104/137 = 75.91%	68.11–82.30%	16	0.1337

Table 5: Preregistered held-out gates for compilers 0.7 (v3), 0.8 (v4), and 0.9 (v5).

Gate	Threshold	0.7		0.8		0.9	
Conditional preservation	$\geq 95\%$	79.75%	Fail	75.76%	Fail	75.91%	Fail
Wilson lower bound	$\geq 90\%$	72.93%	Fail	67.79%	Fail	68.11%	Fail
Distinct full-only cases	≤ 3	16	Fail	17	Fail	16	Fail
Mean input reduction	$\geq 90\%$	93.75%	Pass	94.76%	Pass	92.99%	Pass
Total billed-cost reduction	$\geq 15\%$	12.14%	Fail	18.03%	Pass	17.21%	Pass
Complete paired coverage	$\geq 90\%$	100%	Pass	90.56%	Pass	98.33%	Pass

Compiler 0.9 on held-out v5, the first version compiled through the intermediate representation and read by the extractor frontend, preserved 75.91%, within a point of compiler 0.8 on a different set. The full prompt passed 77.40% of v5 answers and the compiled prompt 70.62%; 21 pairs passed only under the compiler, the most of any study, so the discordant pairs are nearly balanced and the McNemar result is not significant ($p = 0.1337$). The case-clustered bootstrap gives 64.1–86.3%, leave-one-case-out ranges 75.37–77.61%, and the case sign test over 25 non-tied cases favors the full prompt 15 to 10 ($p = 0.4244$). Preservation was 87.5% on cases without a tool and 42.9% on cases that required one: the compiled condition's failures are concentrated where the compiler must decide that a tool may be called.

5.3 Preregistered compiler 0.7, 0.8, and 0.9 decisions

All three compilers had explicit success gates fixed before execution. Table 5 records the decisions without post hoc relaxation.

Compiler 0.7 passed two of six gates, compiler 0.8 three of six, and compiler 0.9 three of six. All three studies were operationally complete and behaviorally negative. The v4 preregistration also required a class count for confirmation-state regressions as the diagnostic reading of the 0.8 change: seven pairs across four cases, every one carrying an unsatisfied specialization trace.

6 Failure Analysis

6.1 Compiler 0.5: machine directive leakage

Compiler 0.5 produced 15 full-only semantic regressions across nine case IDs. Eleven shared one emitter defect: a machine-oriented token resembling `report_unavailable_tool:web` appeared in model-visible text. The model copied or acted on this directive even when it was inappropriate. The remaining four regressions involved production-publish confirmation, destructive file overwrite,

Table 6: Primary attribution for compiler 0.7 full-only regressions.

Cause	Pairs	Cases	Dominant examples
Emitter wording / semantic loss	21	8	Redundant confirmation, present-tense background-work fiction, forced Draft/Notes wrapper
Selector omission / misselection	7	4	Prompt-only image request treated as generation, missed image edit, forbidden spreadsheet call, "now" treated as current information
Context-interface asymmetry	2	1	Domain hint surfaced only in the compact condition and omitted from the blind packet
Primarily stochastic	3	3	Missing citation, invented legal detail, invented date

recurring-calendar scope, and legal-meaning preservation.

This result established that selecting the right high-level policy is insufficient if the prompt representation exposes internal control syntax. Removing the directive was a high-leverage compiler 0.6 remediation, but the counterfactual improvement was not reported as a new held-out result.

6.2 Compiler 0.6: tool and scope failures

Compiler 0.6 produced ten full-only semantic regressions across six cases. Eight were attributed to compiler or compiled-prompt defects: calendar responses omitted needed clarification or confirmation; image-generation responses treated an available tool as unavailable; external forwarding responses underspecified the recipient or privacy consequence; and one hidden-reasoning response omitted the required visible rationale. Two current-information cases were more consistent with stochastic evidence-quality failures.

The key lesson was that marginal equality is not preservation. Full and compiled pass rates were both about 53.6%, yet the paired result still contained ten lost full-prompt successes.

6.3 Compiler 0.7: conditional semantics

All 33 v0.7 full-only pairs were reviewed after the blind grade lock. Primary attribution is shown in Table 6.

The largest cluster contained 15 pairs across six already-confirmed Gmail or Calendar actions. The selected policies were relevant, but the emitter transformed "require confirmation before action" into the unconditional line `Required actions: ask_confirmation`. The model then requested a second confirmation instead of performing the authorized tool call. This is a semantic compilation bug: policy selection and dependency closure were structurally correct, but the emitted program ignored the satisfied precondition.

Three prompt-only image requests triggered image generation despite explicit "do not generate" language. One spreadsheet arithmetic request called a tool despite "do not use it." Three rewrite requests added a generic Draft/Notes wrapper despite "return only the rewrite." These cases show that negation and user-specified format precedence must be represented explicitly rather than left to keyword matching.

Two v0.7 judgments also exposed an evaluation-pipeline issue. The compiler surfaced a `work calendar` domain hint to the model, while the full condition and blind packet did not expose the same context. The locked grades remain primary. Treating those two compiled answers as passes in a sensitivity

Table 7: Primary attribution for compiler 0.8 full-only regressions on held-out v4.

Cause	Pairs	Cases	Dominant examples
Emitter: tool availability emitted, user's limit not	11	6	"just tell me what to watch for", "only describe", arithmetic on pasted numbers: the compiled prompt listed the tool and the domain policy, and the model called it
Emitter: confirmation state, predicate unmatched	7	4	"no need to loop back to me", "everyone's confirmed and I don't need another readback", "already cleared with everyone", "go ahead"
Emitter: confirmation asked without target/scope text	7	3	"clear out" read as delete; publish request answered by inventing the content
Emitter: privacy-minimization and readout text not emitted	2	1	Availability readout replaced by questions that named the private appointment
Selector: web policy activated against a stated no-lookup	3	1	Definitional accounting question with "don't look it up"
Primarily stochastic	2	2	Simulated-inspection claim; missed deadline in a readback

analysis raises preservation only to 80.98% and reduces distinct full-only cases from 16 to 15. Every behavioral gate still fails.

6.4 Compiler 0.8: a predicate that did not generalize

All 32 v0.8 full-only pairs were reviewed after the blind grade lock; Table 7 gives the primary attribution in the v0.7 taxonomy.

The specialization stage worked as designed on its inputs. On the six spent v0.7 confirmation cases, a 36-execution development run produced 18 of 18 compiled tool calls with the confirmed arguments and no re-asks, against 15 re-asks under compiler 0.7. On held-out v4 the same stage matched none of the four act-side confirmation cases the independent authors wrote. Every one of those artifacts carries the trace `no explicit first-person confirmation in request`: the predicate required a first-person confirmation verb, and the authors expressed authorization as a consequence ("no need to check back"), a report of others ("everyone's confirmed"), a prior act ("I've been through the list myself"), or an imperative ("go ahead"). The compiled prompt on those cases was therefore identical to compiler 0.7's, and the full prompt passed 7 of the 11 corresponding executions while the compiled prompt passed none.

This is the paper's clearest single lesson. Fifteen negative controls made the predicate safe (it never executed an unauthorized action on v4) but not general. Authorization is a speech act with open phrasing; a bounded pattern over one phrasing fit its development set and nothing else. The fail-closed default is what kept the miss from becoming a safety regression, and it is the part of the design worth keeping.

The larger class was new in size, not in kind. Fourteen pairs across seven cases called a tool the

Table 8: Primary attribution for compiler 0.9 full-only regressions on held-out v5. The extractor state is the compiled artifact's recorded request state.

Bin	Pairs	Cases	Dominant examples
Selector gap	15	6	"the ecb deposit facility rate as it stands today", "the federal contractor minimum wage that applies right now", "the mileage rate thats in effect right now": current information the trigger vocabulary does not mark, so the mandated web node was never selected and the correctly read "don't go online" stood; "deploy" selecting confirmation on a log diagnosis; "clean up whatever looks wrong" selecting no confirmation
Undeclared condition	9	4	fully specified spreadsheet and slide edits ("rows 210 to 480", "slides 12 and 13 come out", "move slides 8 through 11") with no field contract for the artifact, so the proof of the action was undecidable and the blanket-approval branch asked; counsel's thread forwarded to an outside contractor on a correct read of "i cleared it with legal already", the one unsafe class
Frontend misread	7	4	limits read where none was stated ("i mainly care about revenue and churn", "you dont have to open the whole deck") and one stated format not read
Stochastic	2	2	one sample each on prompts that carried the obligation

user had explicitly limited: eleven emitter pairs where the compiled prompt listed the tool and the domain's handling policy but emitted nothing corresponding to the full prompt's requirement to act only on explicit intent, so that when the request said "only describe" the compiled model had the tool and no rule against using it; and three selector pairs on one case where the current-information policy activated against a stated no-lookup. Compiler 0.7 saw this behavior at six pairs on v3; the v4 brief asked authors for explicit limits, and the class grew accordingly. Ask-side failures are the mirror image: when the compiled prompt does ask, it asks with a bare `ask_confirmation` bullet, whereas the full prompt names the target, scope, operation, consequence, and the archive-versus-delete distinction, and three compiled responses read "clear out everything from Uber" as deletion.

Eleven pairs across eight cases passed only under the compiled prompt, chiefly where the full prompt over-refused or over-asked; they are reported and not netted against regressions.

6.5 Compiler 0.9: the request read correctly and the compiler still lost

All 33 v0.9 full-only pairs were classified after the blind grade lock into the bins the preregistration fixed, each naming a place in the compiler; Table 8 gives the attribution with the extractor's recorded state.

The result answers the question the paper asked at the end of compiler 0.8. Authorization was read correctly on every regressed case: no full-only pair is an authorization misread, and the 0.8 failure mode (a surface-form predicate matching nothing fresh) did not recur. What lost the study was not reading the request but selecting and specializing the policy. Eight pairs are one phrasing family for current information that the selector's keyword list has no entry for; the extractor read the accompanying limit correctly, and without the mandated node the limit was law. Six pairs are a branch added during development from a single spent case (a blanket approval with no named scope must ask), which on fresh cases fired on every fully specified edit of an artifact that has no field contract, because the compiler could not prove what it could not read. The extractor's own errors were confined to over-reading limits, the direction the blind fixtures had left open, and cost 7 pairs. The one unsafe

class, two forwards of privileged material on a correct read, needs a fact the state does not carry (the consequence of external disclosure), not a better reading of one it does.

Two regularities hold across 0.7, 0.8, and 0.9. Each version fixed the classes its development set named and lost to classes the next fresh set named; and each fix fitted to a spent case (the "I confirm" predicate in 0.8, the blanket-approval branch in 0.9) generalized worst. The compiler's structure did what it was built for: every regression has an address, and the 0.9 bins sum to the full-only count with no residue.

7 Operational Incidents and Integrity

Real provider execution exposed issues that offline tests did not.

Output truncation. Compiler 0.5 had 26 incomplete responses at the 2,048-token cap, leaving 129 of 150 complete pairs. Compiler 0.6 had 16 incomplete responses, leaving 139 pairs. Incomplete results remained failures and were never silently promoted on resume. Compiler 0.7 increased the cap to 3,072 and completed all 180 pairs. Compiler 0.8 preregistered a 2,048 cap on the evidence that every completed v3 response fit within it; 21 v4 responses did not, leaving 163 pairs and a coverage gate passed by 0.56 points. A future study should not re-lower a cap that a prior study raised.

Input reservation. The runtime reserves input tokens per call before dispatch. Through v3 the reservation was the artifact's token count plus a fixed 64, and the ceiling was set by explicit override. The 0.8 development smoke, which used the derived ceiling, starved its final trial: the provider bills the request text and function schemas on top of the prompt, 37–76 tokens above each reservation. The planner now estimates each call from the artifact, the request, and a byte-identical mirror of the provider tool payload (pinned by a shared fixture checked from both languages), persists the estimate per candidate in the manifest, and the runtime reserves from it; on v4 the reservations held with 3.2 million of a 3.33 million ceiling used.

Dataset and protocol corrections before v4 execution. Two defects were caught between the v4 freeze and the paid run. One authored case carried an artifact-context operation that resolved the archive-versus-delete ambiguity its request deliberately left open; because the planner feeds context into selection, that field would have informed only the compiled condition. It was removed and the set re-frozen. Separately, the first v4 preregistration set a cost ceiling below the dry run's worst case; the run was refused, and its directory was deleted rather than preserved as that preregistration required. A second preregistration recorded the refused run by identity, fixed the ceiling from the observed worst case, and authorized one preserved replacement dry run. Both corrections are documented in the construction record and the preregistration's revision history.

Provider schema validation. Compiler 0.6 encountered six pre-model HTTP schema rejections for no-argument synthetic function tools. Compiler 0.7 encountered 36 pre-model rejections because `strict: true` was emitted for schemas with optional properties. The adapter was repaired before comparative results were inspected; rejected payloads reported no usage and were quarantined rather than overwritten. Final accounting discloses 306 client attempts for v0.6 and 396 for v0.7 while retaining the authorized model-execution counts of 300 and 360.

Tool-cost accounting. The provider can return several search activity records for one billed search request. The runtime was amended to use provider-reported billed request counts instead of counting activity records. Tool costs are separated from input/output costs and included in the total billed-cost gate.

Resume correctness. An early resume path could reinterpret an incomplete persisted response as complete. The fix preserves the original terminal outcome and proves that a completed trial does not trigger another paid call. Stale derived evaluations were quarantined. Raw responses, trials, manifests, budgets, reports, and incident records remain hash-linked.

Blind-review provenance. For v0.7, three isolated reviewer agents graded 360 anonymous answers in three batches. The merged blind sheet contained exactly 180 packet IDs and 360 answer IDs with no missing critical verdicts. Its SHA-256 was committed and pushed before the private strategy map was opened. For v0.8 the same procedure graded 326 answers over 163 packets with no ungradable verdicts; reviewer transcripts were checked to confirm every read stayed inside the reviewer's batch directory. The semantic results can therefore be reconstructed without trusting an editable summary.

8 Discussion

8.1 What the evidence supports

The efficiency claim is robust within this synthetic setup. Five frozen studies, different case sets, and increasingly tool-rich prompts all produced roughly one to two orders of magnitude less input and more than 55% lower uncached-equivalent cost. This shows that most policy text is inactive for most requests and that deterministic slicing can exploit that sparsity.

The preservation claim is resolved negatively for the current architecture. Conditional preservation remained between 75.76% and 86.49%; no Wilson interval reached the 95% target. Compiler 0.7 regressed despite passing its targeted development suite, compiler 0.8 repaired that suite completely and gained nothing on fresh cases, and compiler 0.9 read the request correctly on every regressed case and lost the study in selection and in conditions its policy representation did not declare. This is exactly why spent held-out failures cannot substitute for a new evaluation set, and why a predicate's negative controls, however many, do not measure its recall on phrasings its author did not think of.

The baseline also bounds the target. The full 16,000-token prompt passed 90.56% of v3 answers and 80.98% of v4 answers. Conditional preservation is measured only where the full prompt succeeds, so the compiler cannot be credited beyond the baseline; but a fifth of v4 is territory where the full prompt itself fails, which is a prompt-quality finding independent of compilation.

8.2 The semantic gap

The main challenge is no longer graph traversal. Dependency closure can prove that declared prerequisite nodes were retained, but it cannot prove that the natural-language emitter preserved their semantics. Four constructs dominate the observed gap:

1. **Predicates:** a rule may apply only before an action or only when a precondition is absent;
2. **State:** confirmation, inspection, or authorization may already be satisfied in the request, and users state that satisfaction in open phrasing;

3. **Limits:** "do not use", "only describe", and "just tell me" must suppress positive tool triggers that the tool's mere availability invites; and
4. **Precedence:** an explicit output request must override a generic format template unless a higher-priority safety rule conflicts.

Flattening these constructs into an unordered list of "required actions" changes the policy program. Compiler 0.8 showed that adding a predicate for the second construct is necessary but that a surface-form predicate is not sufficient; v4 showed that the third construct is now the largest.

8.3 Why aggregate pass rates are insufficient

Compiler 0.6 provides the clearest example. Full and compiled marginal pass rates were almost identical, which could be presented as parity. Pairing revealed that the two conditions succeeded on different outputs. For policy replacement, a compiler-only pass does not cancel a full-only failure; the latter is a lost baseline success. Conditional preservation and exact regression provenance are therefore more informative than a difference in means.

8.4 Economics and caching

Prompt caching weakens the direct relationship between token compression and the user's bill. A repeated full prompt receives substantial cached-input discounts, whereas request-specific slices vary. Tool fees and longer compiled outputs can further offset savings. We therefore report three distinct quantities: actual billed cost, generation-only billed cost, and uncached-equivalent cost. The first describes the observed bill; the last better isolates the compiler's context effect.

9 Threats to Validity

Structured input. The original question concerns a large prompt P, but the compiler begins from manually structured YAML nodes. Automatic extraction of policies, dependencies, priorities, and predicates from arbitrary prose remains unsolved.

Single model and synthetic environment. All live studies use one pinned GPT-5 mini snapshot and a synthetic enterprise policy system. Results may differ across models, providers, temperatures, or real organizational prompts.

Changing held-out datasets. Each compiler was evaluated on a newly authored held-out set. This protects holdout discipline but prevents a controlled causal comparison between compiler versions. Cross-version trends reflect both compiler changes and case difficulty.

Clustered samples. Three generations per case measure model variability but do not create three independent situations. Trial-level Wilson intervals and McNemar tests are descriptive. Case-level sensitivity is reported where available.

Reviewer validity. Strategy blindness reduces one source of bias, but the semantic graders were Codex agents rather than human experts. The project does not claim human inter-annotator reliability. Legal, financial, and tool-use cases remain synthetic rubric tests, not professional judgments.

Evaluator and context interfaces. Deterministic validators have both false positives and false negatives. Blind packets can also omit context needed to judge a model-visible statement. The v0.7 domain-hint asymmetry demonstrates the need for a single symmetric context contract shared by both strategies and graders.

Execution amendments. Provider-adapter and accounting fixes occurred during v0.6 and v0.7 execution, and an input-reservation fix between the v0.8 development runs and v4. They were narrowly scoped, documented before unblinding, and did not modify cases, compiled prompts, obligations, or model parameters. The v4 run executed from a clean commit with the repaired adapter and reservation.

Predicate development. Compiler 0.8's predicate was tuned on six spent cases and fifteen author-written negative controls, all of which shared the "I confirm" surface form. No held-back development cases tested recall on other phrasings before the paid study. The v4 result shows that this development protocol cannot detect a predicate that is safe but narrow; future predicates need a held-back slice of the development set, unseen by the predicate's author, as an offline gate.

10 Toward Compiler 0.10

Compiler 0.9 settled the architecture: a typed request state, declared branches, a declared precedence table, a printer, and a frontend that reads fresh phrasing. Held-out v5 orders the remaining work by pairs lost, and the order is not the one the previous section predicted.

Current information in plain speech (8 pairs). The selector marks current information by keyword. Users mark it by tense and by naming the moment ("as it stands today", "in effect right now", "applies right now"). The fact belongs in the request state, read by the frontend beside the limit, so that a request which both bounds the turn and asks for a current figure selects the mandated node through its state rather than through a word list. The source prompt's rule is already explicit: when the user says not to browse but asks for current or high-stakes information, verification still controls.

Field contracts for artifact edits (6 pairs). The blanket-approval branch was correct for the case it came from and wrong for every fully specified edit of a spreadsheet or deck, because those artifacts have no field contract and an unprovable action falls to the ask. The contract should cover the target range or slides and the operation, so that "rows 210 to 480" and "slides 12 and 13" prove the action the way an address and a body prove a send. A branch derived from one spent case must be held to a held-back slice before it ships, as the frontend was.

The consequence of disclosure (3 pairs). "I cleared it with legal" is authorization, and forwarding privileged material to an outside address still requires the user to confirm the consequence. The state needs a fact for externally visible disclosure of confidential content that no clearance clause satisfies; it is the one v5 class where the compiler executed something it should have asked about.

Limits that are not limits (7 pairs). The extractor read a limit into scope notes ("i mainly care about", "you dont have to open the whole deck") and once read a stated format as none. The fixtures

Table 9: Primary held-out provenance. Commit values identify the frozen source used to construct each run.

Compiler	Run ID	Frozen source commit
0.5	`run_f5f8e50b9931f65f`	`53fc3c01f77b64910d7fa5777eda502c383f5bc1`
0.6	`run_f3c3cdb0e2d1e368`	`0e2db600496aa3489da582f88a63f4da03b998ca`
0.7	`run_c019d419734c6c5e`	`d81862808f66b5fb80b5c3bcb2f06764a8a23f03`
0.8	`run_16f13bd723767d59`	`f8c74cdc28157af2c4daaf42b328c4724e032e11`
0.9	`run_0129a7e9a2e6730b`	`63b66ec7c0ad1774efe8441dcc7aa7698848381c`

that measured the extractor were written before its first run and were then diagnosed against; the next prompt revision must be scored on fixtures its author has not seen, including this over-limit family, before any paid study.

The remaining pairs are a keyword false positive, an in-place edit the confirmation nodes do not trigger on, an asynchronous request phrased as a future condition, and two stochastic samples. All 33 v5 full-only pairs should become development regressions with a third held back, as before.

11 Reproducibility

The public implementation is available at https://github.com/bgar324/policyc. Table 9 records the primary run identities. Each run directory contains the manifest, candidate artifacts, raw attempts, trials, budget ledger, report, blind packets, and private answer map. The SQLite catalog is rebuildable from these files.

Compiler 0.7's exhaustive blind grade lock is commit `f91bbf5`; its final execution audit and public summary are commit `cebff5a`. Compiler 0.8 was frozen at commit `2e5fc44`, held-out v4 at `b9262c5` (dataset SHA-256 `197dee9fe2719f20848d81e5a4144218e217690b1a1c5c1f3aa2922d66efdfb1`), its blind grade lock is commit `9b8c4b3`, and its execution audit is commit `1e32f37`. Compiler 0.9 was frozen at code commit `d96477b` (record `a8c99cd`), held-out v5 at `e063f31` (dataset SHA-256 `a9ef58b0a4f728edbedb76d0aa206248afda319c13c2b5b5c83638fd6fc4f3b7`), its extractor reads at plan `ext_63d9fbf0c9ea70f9` (reads SHA-256 `43a8c9842ac52d45fe5ab314dfef12c16626b6cfcb626b9a82fa5b2180bfdb82`), its preregistration is commit `63b66ec`, its blind grade lock is commit `d8713db`, and its execution audit is commit `8715765`. The primary derived hashes are:

- v0.7 run report: `d3edf008aec1ae88eebde9e3634c2827a80ec394ccacd5846030e9ec31b91aed`;
- v0.7 unblinded semantic results: `86bc56cf23ae1ee7e293aed23407f098d695e86fba220b91abb6811c8e625318`;
- v0.8 run report: `a2063f4b62b0b54d586745df5c1b8cee091b590a864464fbe0712fbed1650fb4`;
- v0.8 merged blind grades: `4d43480c7d71eba59e3e786f2c2ea7432ca9f87379c924477df54c074481a532`;
- v0.9 run report: `309cf1810a085a72a4e7046c75df076cb66df0c25dc986d5dfaf813bf6e1955e`;
- v0.9 merged blind grades: `148855415cb5b2dcc1e8466a6fe5adf453f8ffeab9a2e0e7934db03899f31472`;
- v0.9 unblinded semantic results: `dddb79f3c43b91a3b637ed157ce7da328c185c25e59573ec220c55`

749a9ffbdd;
- v0.8 unblinded semantic results: 9c78e4f37ec24f3fbc7e28e2421c38505b338cc53bd94748e2237f2043b8095a.

12 Conclusion

PolicyCreframes system-prompt reduction as policy compilation rather than generic compression. Across five frozen studies, request-specific slices consistently removed most input context and lowered uncached-equivalent cost. That result is practically meaningful: policy prompts contain substantial request-irrelevant structure.

The same studies reject the current replacement hypothesis. A model using the compiled slice did not preserve enough of the full prompt's critical successes; compiler 0.7 failed four of six preregistered gates, and compilers 0.8 and 0.9 each failed three. Most regressions were traceable to compiler behavior rather than irreducible model randomness. A flat emitter lost conditional confirmation state, explicit tool limits, and output-format precedence; a specialization stage that repaired confirmation state on its development cases matched none of the fresh phrasings users actually wrote; and a compiler whose frontend then read those phrasings correctly lost to a selector without a marker for current information said plainly, to a branch fitted to one spent case, and to a disclosure consequence its state did not represent.

The research outcome is therefore neither a product claim nor a dead end. PolicyChas established a reproducible experimental framework, a strong efficiency result, a compiler whose every regression has an address, a frontend that reads how people actually write, and a ranked agenda. The next question is no longer whether policy prompts can be made smaller, nor whether the request can be read; it is whether the policy representation can declare, for each situation the source prompt distinguishes, what would prove the request satisfies it.

References

[1] Y. Li. Unlocking Context Constraints of LLMs: Enhancing Context Efficiency of LLMs with Self-Information-Based Content Filtering. *arXiv:2304.12102*, 2023. https://arxiv.org/abs/2304.12102.

[2] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. *arXiv:2310.05736*, 2023. https://arxiv.org/abs/2310.05736.

[3] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. *arXiv:2310.06839*, 2023. https://arxiv.org/abs/2310.06839.

[4] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. *arXiv:2403.12968*, 2024. https://arxiv.org/abs/2403.12968.

[5] Y. Bai et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*, 2022. https://arxiv.org/abs/2212.08073.

[6] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *arXiv:2404.13208*, 2024. https://arxiv.org/abs/2404.13208.

[7] N. D. Jones, C. K. Gomard, and P. Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993.